A Revolução industrial

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

Contexto Histórico

- Tudo começou no chão de fábrica, nas indústrias;
- Antes da adoção das práticas Lean, o tempo médio para a execução de um pedido em fábricas era de seis semanas;
- Apenas 70% dos pedidos eram entregues no prazo;
- O modelo tradicional de produção gerava desperdício, processos ineficientes e altos custos.

Implementação das Práticas Lean

- Inspirado pelo Sistema Toyota de Produção, o Lean Manufacturing revolucionou a indústria;
- O tempo médio de produção foi reduzido para menos de três semanas;
- A taxa de pedidos entregues no prazo aumentou para 90–95%.

Impacto no Mercado

- Empresas que não adotaram o Lean perderam competitividade;
- Muitas organizações faliram ou saíram do mercado devido à ineficiência.

Consequências da Falta de Adaptação

- Empresas que não implementaram práticas eficientes perderam espaço no mercado;
- Exemplo: A General Motors enfrentou falência em 2009 antes de reformular seus processos;
- A Kodak, mesmo sendo pioneira na fotografia digital, manteve processos rígidos e ineficientes, perdendo relevância para concorrentes mais ágeis;
- A Nokia, que já dominou o mercado de celulares, não conseguiu adaptar sua estrutura de produção à nova realidade do setor, sendo superada por Apple e Samsung.

Evolução das Práticas Lean

- A partir dos anos 1980, o sucesso do Lean Manufacturing consolidou essa filosofia como um diferencial competitivo;
- Empresas ocidentais começaram a implementá-lo de forma transversal, buscando maior eficiência e produtividade.

Objetivos do Lean Manufacturing

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

Objetivos do Lean Manufacturing

- Eliminar desperdícios;
- Otimizar processos;
- Reduzir custos;
- Impulsionar a inovação;
- Diminuir o tempo de colocação de produtos no mercado.

Benefícios do Lean Manufacturing

- Aumento da produtividade e eficiência;
- Manutenção da qualidade dos produtos e serviços;
- Maior segurança no ambiente de trabalho.

Exemplos de Sucesso

- Empresas como a Toyota adotaram o Lean com o Sistema Toyota de Produção;
- O modelo tornou-se referência global na redução de desperdícios e melhoria contínua.

A Evolução da Computação e da Infraestrutura

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

1970–1980: Era dos Mainframes e Computação Centralizada

- Computação dominada por mainframes (IBM, DEC);
- Modelos cliente-servidor ainda não eram comuns;
- Sistemas operacionais proprietários e baixa conectividade entre sistemas;
- Desenvolvimento de software mais lento, ciclos longos e pouca automação.

1970–1980: Era dos Mainframes e Computação Centralizada

- Benefícios:
  - Sistemas altamente confiáveis para operações críticas;
  - Segurança reforçada, pois os dados eram centralizados;
  - Estabilidade e previsibilidade nos processos de TI.
- Desafios:
  - O desenvolvimento de novos recursos levava cerca de cinco anos e custava milhões de dólares;
  - Alto custo operacional, dificultando a inovação.

1980–1990: Computadores Pessoais e Redes Locais

- Popularização do PC (IBM PC, Apple Macintosh);
- Introdução das primeiras redes locais (LANs);
- Surgimento dos primeiros bancos de dados relacionais (Oracle, SQL Server);
- Os softwares começam a ser mais modulares, mas ainda com ciclos longos de desenvolvimento.

1980–1990: Computadores Pessoais e Redes Locais

- Benefícios:
  - Acesso mais fácil à computação para empresas e usuários individuais;
  - Aumento da produtividade com interfaces gráficas mais intuitivas;
  - Início da descentralização dos sistemas.
- Desafios:
  - Os softwares ainda eram monolíticos, com processos de desenvolvimento demorados;
  - Infraestrutura dependente de hardware físico, dificultando a escalabilidade.

1990–2000: A Revolução da Internet e a Era Cliente-Servidor

- A internet (WWW) populariza o acesso à informação e aplicações online;
- Modelos cliente-servidor dominam o mercado, melhorando a conectividade;
- O Linux cresce como alternativa aos sistemas proprietários;
- Arquitetura monolítica ainda predominante nos sistemas empresariais.

1990–2000: A Revolução da Internet e a Era Cliente-Servidor

- Benefícios:
  - Internet amplia a comunicação e a colaboração global.
  - Surgimento de padrões abertos facilita a interoperabilidade entre sistemas.
  - Início da automação no desenvolvimento e gerenciamento de infraestrutura.
- Desafios:
  - Mesmo com avanços, a implementação de novos recursos ainda levava semanas ou meses.
  - Lançamentos de software ainda eram lentos e caros.

2000–2010: Virtualização e a Ascensão da Nuvem

- Virtualização (VMware, Xen, KVM) impulsiona a eficiência dos servidores;
- Amazon Web Services (AWS) lança os primeiros serviços de computação em nuvem (EC2, S3);
- Crescimento da arquitetura SOA (Service-Oriented Architecture);
- Surgem metodologias ágeis (Agile Manifesto - 2001) e CI/CD (Continuous Integration/Continuous Delivery);
- Infraestrutura começa a se tornar mais dinâmica, mas ainda dependente de processos manuais.

2000–2010: Virtualização e a Ascensão da Nuvem

- Benefícios:
  - Virtualização permite melhor utilização dos recursos de hardware;
  - Computação em nuvem reduz custos e melhora a escalabilidade;
  - Adoção de metodologias ágeis reduziu o tempo de desenvolvimento para semanas ou meses.
- Desafios:
  - A implementação na produção ainda era lenta e, muitas vezes, o resultado final era insatisfatório;
  - Falta de automação completa na gestão de infraestrutura.

2010–2020: Microserviços, Containers e DevOps

- Popularização da arquitetura de microserviços para escalar aplicações;
- Surgimento do Docker (2013) e da orquestração de containers com Kubernetes (2014);
- Adoção massiva de CI/CD, IaC (Infrastructure as Code) e DevOps.
- Automação de infraestrutura com Terraform, Ansible, Chef e Puppet;
- Observabilidade se torna essencial (Prometheus, Grafana, ELK Stack).

2010–2020: Microserviços, Containers e DevOps

- Benefícios:
  - DevOps transforma a forma como empresas criam e implementam recursos;
  - Startups conseguem criar empresas inteiras em semanas;
  - Infraestrutura como código permite provisionamento automatizado e ágil;
  - Desenvolvimento e implantação contínua possibilitam lançamentos diários.
- Desafios:
  - Necessidade de cultura organizacional alinhada para adotar DevOps com sucesso;
  - Complexidade no gerenciamento de arquiteturas distribuídas.

2020 até Hoje: Cloud-Native, Inteligência Artificial e Edge Computing

- As arquiteturas Cloud-Native se tornam padrão (AWS, GCP, Azure);
- Inteligência Artificial e Machine Learning Ops (MLOps) começam a ser incorporados ao DevOps;
- Serverless Computing e Function as a Service (FaaS) reduzem a necessidade de gerenciar infraestrutura;
- Edge Computing ganha espaço para processamento distribuído (IoT, 5G);
- Plataformas No-Code/Low-Code aceleram o desenvolvimento;
- Plataformas GitOps e automação cada vez mais avançada.

2020 até Hoje: Cloud-Native, Inteligência Artificial e Edge Computing

- Benefícios:
  - Automação extrema e entrega contínua permitem inovação rápida;
  - Inteligência Artificial ajuda na análise de dados e decisões automatizadas;
  - Serverless e Edge Computing reduzem custos e melhoram a performance;
  - Organizações conseguem experimentar e validar ideias rapidamente, otimizando valor para clientes.
- Desafios:
  - Segurança e conformidade em ambientes distribuídos e altamente automatizados;
  - Necessidade de aprendizado contínuo devido à rápida evolução tecnológica.

O Resultado da evolução da tecnologia

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

Por que a tecnologia evoluiu?

- Para atender as demandas de negócio;
- Organizações que realizam experimentações e transformam ideias em produtos geram mais valor para os clientes.

E por que toda empresa deveria adotar práticas DevOps?

- Empresas que adotam princípios e práticas de DevOps conseguem entregar alterações continuamente e de forma rápida;
- A vantagem competitiva exige um curto tempo para comercializar e experimentar novas soluções;
- Empresas que não adotam DevOps podem perder espaço para concorrentes mais ágeis, similar ao que ocorreu na Revolução Industrial com o movimento Lean.

Exemplos de disrupção tecnológica

- Táxis perderam mercado para aplicativos como o Uber;
- Locadoras de filmes foram substituídas por serviços de streaming como Netflix, Amazon Prime e Disney Plus.
    - Os cinemas estão indo pelo mesmo caminho.

A tecnologia deve estar no centro dos negócios

- No passado, a tecnologia era vista apenas como custo, mas essa percepção mudou;
- Hoje, todas as empresas dependem da tecnologia para adquirir e entregar valor aos clientes;
- Empresas que colocam o software no centro do negócio têm mais chances de sucesso;
- O livro Manual DevOps menciona: "Toda indústria e empresa que não está colocando software no centro do seu negócio será abalada.";
- Exemplos históricos mostram que não podemos ignorar o avanço tecnológico.

Dívida técnica

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

O que não mudou

- A tecnologia mudou e irá continuar mudando;
- Apesar da adoção de novas práticas, novas tecnologias, muitas empresas ainda têm problemas de gestão impactam diretamente na área de TI;
- A dívida técnica acompanha a evolução tecnológica e é algo que devemos nos preocupar.

O que é dívida técnica?

- A dívida técnica é uma metáfora utilizada na engenharia de software para descrever o custo adicional que uma equipe enfrenta no futuro devido a escolhas de design ou desenvolvimento de baixa qualidade no presente;
- Exemplo:
  - Uma dívida financeira aumenta com o acúmulo de juros;
  - Ao longo do tempo, a dívida técnica, se não for tratada, pode aumentar a complexidade e o custo de manutenção do software.

Causas Comuns da Dívida Técnica

- Pressão por Prazos: Decisões apressadas para cumprir prazos podem resultar em soluções subótimas;
- Falta de conhecimento ou experiência: Desenvolvedores podem não estar cientes das melhores práticas ou padrões;
- Requisitos Mal Definidos: Início do desenvolvimento sem uma compreensão clara dos requisitos;
- Falta de Documentação: Correções ou implementações sem documentação adequada;
- Componentes Fortemente Acoplados: Arquiteturas que dificultam a modificação ou integração de novos componentes.

Impactos da Dívida Técnica

- Manutenção Complexa: Aumento da dificuldade e do tempo necessário para manter o código;
- Redução da Qualidade do Software: Mais bugs e falhas devido a soluções inadequadas;
- Dificuldades na Escalabilidade: Impedimentos para crescimento ou adaptação do sistema;
- Moral da Equipe: Frustração entre os desenvolvedores devido a código de baixa qualidade ou difícil de trabalhar.

Gerenciamento e Mitigação da Dívida Técnica

- Priorização: Avaliar quais dívidas têm maior impacto e devem ser tratadas primeiro.
- Refatoração Contínua: Incorporar melhorias incrementais no código durante o desenvolvimento regular;
- Documentação Adequada: Assegurar que todas as mudanças sejam bem documentadas para futuras referências;
- Automação de Testes: Implementar testes automatizados para garantir que alterações não introduzem novos problemas.

Exemplo Prático

- Planejamento Adequado: Investir tempo na definição clara dos requisitos antes de iniciar o desenvolvimento;
- Educação e Treinamento: Manter a equipe atualizada sobre as melhores práticas e novas tecnologias;
- Cultura de Qualidade: Promover a importância da qualidade do código e da arquitetura bem planejada;
- Revisões Regulares: Agendar revisões periódicas para identificar e abordar dívidas técnicas.

O conflito entre as áreas

ROCKETSEAT

Plataforma completa de aprendizado contínuo em programação.

#BoostingPeople

rocketseat.com.br

Todos os direitos reservados © Rocketseat S.A.

Introdução

- Muitos dos problemas de gestão, entrega e confiança na área de TI vem de um conflito histórico;
- Você já deve ter escutado algo como:
  - Hoje vou ser babá de dev;
  - Na minha máquina funciona;
  - A pipe não está funcionando;
  - O sistema está lento;
  - Isso não é comigo;
  - O time de segurança bloqueou;
  - E muitas outras.

Introdução

- Geralmente o time técnico é muito técnico:
- Seja da área de desenvolvimento ou da área de operações.
- O que isso significa:
- Muita preocupação com o como;
- Pouca preocupação com o porquê.

Responsabilidade das áreas

- Time de Desenvolvimento: Foca em responder rapidamente ao mercado, criando novas funcionalidades, produtos e serviços;
- Time de Operações: Foca em manter o serviço estável, confiável e seguro para os usuários finais.

Conflito entre as equipes

- Desenvolvedores precisam entregar código rapidamente e garantir que chegue à produção;
- O time de operações precisa manter a estabilidade e segurança do sistema;
- O desenvolvimento isolado pode resultar em deploys problemáticos, quebrando a aplicação em produção.

Impacto no objetivo da empresa

- O conflito entre os times afeta a meta global da organização;
- Operações, para garantir estabilidade, pode bloquear novos deploys para evitar indisponibilidades;
- Isso prejudica a inovação e a capacidade de resposta rápida ao mercado.